

Core Language Working Group "ready" Issues for the February, 2020 (Prague) meeting

References in this document reflect the section and paragraph numbering of document [WG21 N4849](#).

2053. auto in non-generic lambdas

Section: 9.2.8.5 [dcl.spec.auto] **Status:** ready **Submitter:** Faisal Vali **Date:** 2014-12-07

According to 9.2.8.5 [dcl.spec.auto] paragraph 3,

If the *auto type-specifier* appears as one of the *decl-specifiers* in the *decl-specifier-seq* of a *parameter-declaration* of a *lambda-expression*, the lambda is a *generic lambda* (7.5.5 [expr.prim.lambda]).

and 7.5.5 [expr.prim.lambda] paragraph 5 says,

For a generic lambda, the closure type has a public inline function call operator member template (13.7.2 [temp.mem]) whose *template-parameter-list* consists of one invented type *template-parameter* for each occurrence of *auto* in the lambda's *parameter-declaration-clause*, in order of appearance.

However, an *auto* that signals a *trailing-return-type* should be excluded from these descriptions.

Proposed resolution (February, 2020):

This issue is resolved by the resolution of issue 2447.

2445. Partial ordering with rewritten candidates

Section: 13.7.6.2 [temp.func.order] **Status:** ready **Submitter:** Hubert Tong **Date:** 2020-01-29

The tiebreaker based on partial ordering of function templates should presumably operate upon rewritten candidates based on their parameter lists for the purpose of overload resolution (12.4.1 [over.match.funcs]) without substitution of template arguments instead of the function parameter lists of the templates themselves.

It is observed, however, that neither GCC nor Clang performs partial ordering in the manner described above. In the following case, considering the templates with the reordering should yield 1a as being more specialized than 2; however, the wording is not especially clear about this and both GCC and Clang seems to fall past the partial ordering tiebreaker to pick 2 for the case as-is. If 1b is introduced, then it is more specialized than 2 in either ordering, and it is chosen by both GCC and Clang.

```
template <typename> constexpr bool F = false;
template <typename T> struct A { };

template <typename T, typename U>
// bool operator==(A<T>, A<U *>);           // 1b
bool operator==(T, A<U *>);                 // 1a

template <typename T, typename U>
bool operator!=(A<T>, U) {                  // 2
    static_assert(F<T>, "Isn't this less specialized?");
    return false;
}

bool f(A<int> ax, A<int *> ay) { return ay != ax; }
```

Proposed resolution (February, 2020):

Change 13.7.6.2 [temp.func.order] paragraph 3 as follows:

To produce the transformed template, for each type, non-type, or template template parameter (including template parameter packs (13.7.3 [temp.variadic]) thereof) synthesize a unique type, value, or class template respectively and substitute it for each occurrence of that parameter in the function type of the template. [Note: The type replacing the placeholder in the type of the value synthesized for a non-type template parameter is also a unique synthesized type. —end note] **If only one of the Each function templates template M that is a member function, and that function is a non-static member of some class A, M is considered to have a new first parameter of type X(M), described below, inserted in its function parameter list. Given cv as the If exactly one of the function templates was considered by overload resolution via a rewritten candidate (12.4.1.2 [over.match.oper]) with a reversed order of parameters, then the order of the function parameters in its transformed template is reversed. For a function template M with cv-qualifiers of M (if any), the new parameter cv that is a member of a class A:**

- **The type X(M) is of type “rvalue reference to cv A” if the optional ref-qualifier of M is && or if M has no ref-qualifier and the first positionally corresponding parameter of the other transformed template has rvalue reference type; if this determination depends recursively upon whether X(M) is an rvalue reference type, it is not considered to have rvalue reference type.**
- Otherwise, ~~the new parameter~~ **X(M)** is of type “lvalue reference to cv A”.

[Note: This allows...

2446. Questionable type-dependency of *concept-ids*

Section: 13.8.2.2 [temp.dep.expr] **Status:** ready **Submitter:** Hubert Tong **Date:** 2019-02-03

The rules for type-dependency do not appear to take the `bool` type associated with *concept-ids* into account. For example:

```
template <typename T> concept C = true;
template <typename T> struct A;
template <> struct A<bool> { using type = bool; };

template <typename T>
void f(A<decltype(C<T>)>::type); // error: needs typename to avoid vexing parse
```

Proposed resolution (February, 2020):

1. Change 13.8.2.2 [temp.dep.expr] paragraph 3 as follows:

An *id-expression* is type-dependent if **it is not a *concept-id* and** it contains

- ...

2. Change 13.8.2.3 [temp.dep.constexpr] paragraph 3 as follows:

An *id-expression* is value-dependent if:

- **it is a *concept-id* and any of its arguments are dependent,**
- it is type-dependent,
- ...

2447. Unintended description of abbreviated function templates

Section: 9.2.8.5 [dcl.spec.auto] **Status:** ready **Submitter:** Hubert Tong **Date:** 2019-10-15

The current wording for abbreviated function templates could lead to the incorrect conclusion that `f` and `g` in the example below are well-formed abbreviated function templates. The `g` case is the abbreviated function template analog of issue 2053 regarding generic lambdas.

9.2.8.5 [dcl.spec.auto] paragraph 4 clearly disallows the `ap` case. The inconsistency between the paragraph 2 wording for abbreviated function templates and the paragraph 4 wording is unintentional.

```
template <typename> struct A;
void f(A<auto> x);
void g(auto f() -> int);
A<auto> *ap = static_cast<A<int>*>(0);
```

Proposed resolution (February, 2020):

1. Change 9.2.8.5 [dcl.spec.auto] paragraph 2 as follows:

A *placeholder-type-specifier* of the form *type-constraint*_{opt} `auto` can be used **in as a *decl-specifier of*** the *decl-specifier-seq* of a *parameter-declaration* of a function declaration or *lambda-expression* and, **if it is not the *auto type-specifier* introducing a *trailing-return-type* (see below), is a *generic parameter type placeholder* of the function declaration or *lambda-expression*. [Note: Having a *generic parameter type placeholder*** signifies that the function

is an abbreviated function template (9.3.3.5 [dcl.fct]) or the lambda is a generic lambda (7.5.5 [expr.prim.lambda]). **—end note**

2. Change 9.3.3.5 [dcl.fct] paragraph 18 as follows:

An *abbreviated function template* is a function declaration ~~whose parameter-type-list includes~~ **that has** one or more **generic parameter type** placeholders (9.2.8.5 [dcl.spec.auto]). An abbreviated function template is equivalent to a function template (13.7.6.1 [temp.over.link]) whose *template-parameter-list* includes one invented type *template-parameter* for each ~~occurrence of a~~ **generic parameter type** placeholder ~~type in the decl-specifier-seq of a parameter-declaration in the function's parameter-type-list~~ **of the function declaration**, in order of appearance. For a *placeholder-type-specifier* of the form...

3. Change 7.5.5 [expr.prim.lambda] paragraph 5 as follows:

A lambda is a *generic lambda* if ~~there is a decl-specifier that is a placeholder-type-specifier in the decl-specifier-seq of a parameter-declaration of the lambda-expression~~ **has any generic parameter type placeholders (9.2.8.5 [dcl.spec.auto])**, or if the lambda has a *template-parameter-list*. [Example:...

This resolution also resolves issue 2053.